Complexity, and the Array

DSA27 — Lecture 02 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 2 · Fall 2026

Where We Are

Last week, in one line

An **ADT** is a contract. A **data structure** keeps the contract. Choosing between structures **is the engineering** — and you cannot choose without knowing what each one **costs**.

Today: **how to state a cost**, and **the first structure whose costs we state**.

Today

1. What we count, and why we throw most of it away
2. Big-O, Big-Ω, Big-Θ — the definitions, and one proof
3. Best, worst and average case; time and space
4. Rules for reading the cost off code
5. **The array**: what it is in memory, and what it costs
6. Arrays in Python — and **the course `Array`**
7. **The rule** from today on: no `list`, `dict` or `set` inside a structure

What Do We Count?

Two ways to answer “how fast?”

Measure	**Count**
time it with a clock	count the basic steps
true for this machine, this input, today	true for every machine
gives you the constants	gives you the **shape**
`viz.complexity.measure`	pen and paper

You need both. This lecture is mostly about counting — and ends by measuring.

The model: basic steps

A **basic step** takes a constant amount of time, whatever the input size:

- arithmetic and comparison on small numbers: `a + b`, `x < y`
- assigning a variable: `x = y`
- reading or writing **one** array slot: `a[i]`, `a[i] = v`
- calling a function, returning from one (not what the function then does)

The cost of an algorithm is **the number of basic steps**, as a function of the input size **n**.

Counting a loop

```
def total(arr):
    result = 0                  # 1 step
    for i in range(len(arr)):   # the body runs n times
        result += arr[i]        #   2 steps: read, add-and-assign
    return result               # 1 step
```

$$T(n) = 2n + 2$$

Counting a nested loop

```python
def count_pairs(arr):
    n, pairs = len(arr), 0
    for i in range(n):
        for j in range(i + 1, n):      # runs n-1, n-2, ..., 1, 0 times
            if arr[i] == arr[j]:
                pairs += 1
    return pairs
```

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2} = \tfrac{1}{2}n^2 - \tfrac{1}{2}n$$

Throw away what does not matter

n	$\frac{1}{2}n^2$	$\frac{1}{2}n$	share of the total from $\frac{1}{2}n^2$
10	50	5	91%
1,000	500,000	500	99.9%
1,000,000	5×10^{11}	500,000	99.9999%

For large n, **the fastest-growing term is everything**. And the constant in front of it (½) depends on the machine and on how you count. So we drop both:

$$\frac{1}{2}n^2 - \frac{1}{2}n \quad \longrightarrow \quad n^2$$

Big-O, Big-Ω, Big-Θ

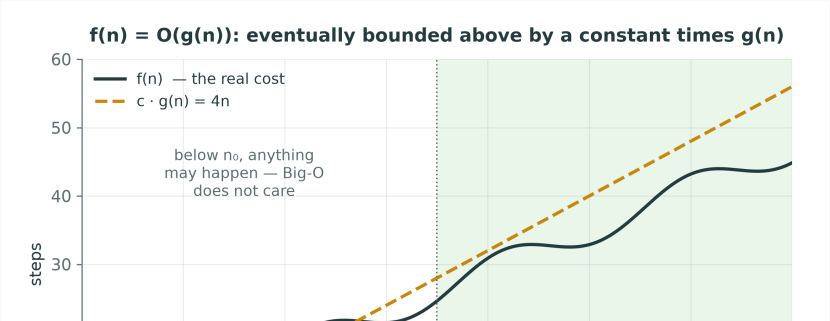
Big-O: an upper bound

$$f(n) = O(g(n))$$

means: there are constants $c > 0$ and n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for every } n \geq n_0.$$

“From some point on, f never grows faster than a constant times g.”



A proof, once

Claim. $3n + 5 = O(n)$.

Proof. Choose $c = 4$ and $n_0 = 5$. For every $n \geq 5$:

$$3n + 5 \;\leq\; 3n + n \;=\; 4n.$$

So $3n + 5 \leq 4 \cdot n$ for all $n \geq 5$. ■

Big-Ω and Big-Θ

Notation	Means	Like
$f = O(g)$	$f \leq c \cdot g$ eventually	$\leq$ — an **upper** bound
$f = \Omega(g)$	$f \geq c \cdot g$ eventually	$\geq$ — a **lower** bound
$f = \Theta(g)$	both: $c_1 g \leq f \leq c_2 g$	$=$ — a **tight** bound

$\frac{1}{2}n^2 - \frac{1}{2}n$ is $O(n^2)$, $\Omega(n^2)$ and therefore $\Theta(n^2)$. It is also $O(n^3)$ — true, and useless.

The classes you will meet

Big-O	Name	Example in this course	n = 1,000,000
$O(1)$	constant	`a[i]`; push onto a stack	1
$O(\log n)$	logarithmic	binary search (Week 8)	20
$O(n)$	linear	linear search; insert into an array	10^6
$O(n \log n)$	linearithmic	merge sort, heap sort (Week 10)	2×10^7
$O(n^2)$	quadratic	bubble, selection, insertion sort	10^{12}
$O(2^n)$	exponential	all subsets (Week 3)	—

Best, Worst, Average

One algorithm, many costs

```python
def find(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Case	When	Comparisons
Best	target is `arr[0]`	1 — $O(1)$
Worst	target is last, or absent	n — $O(n)$
Average	present, every position equally likely	$\frac{n+1}{2}$ — $O(n)$

Measured: same n, different cost

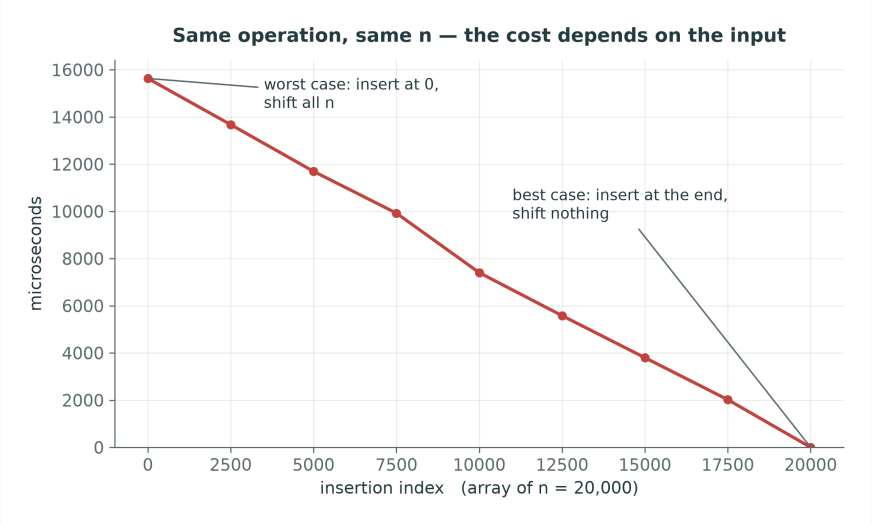


Figure: Insertion into an array of 20,000 elements, at nine different positions. Real timings on the course `Array`.

Time and space

Space complexity: how the *extra* memory an algorithm needs grows with n.

Function	Extra space	Why
`find(arr, x)`	$O(1)$	two variables, whatever n is
`reverse_in_place(arr)`	$O(1)$	swaps within the array
`resized(arr, 2n)`	$O(n)$	a whole new array
`merge_sorted(a, b)`	$O(n+m)$	the output array
recursive `total(values)`	$O(n)$	one stack frame per element

Reading the Cost Off the Code

Five rules

1. **Sequence — add.** $O(f)$ then $O(g)$ is $O(f + g)$ = the bigger one.
2. **Loop — multiply.** A loop of n iterations with an $O(g)$ body is $O(n \cdot g)$.
3. **Nested loops — multiply again.** Two full nested loops over n: $O(n^2)$.
4. **Halving — logarithm.** A loop that halves what is left is $O(\log n)$.
5. **Function calls cost what the function costs** — including built-ins.

Rule 5: the hidden loop

```
def has_duplicate(values):
    for i in range(len(values)):
      if values[i] in values[i + 1:]:    # <- how much does this line cost?
            return True
    return False
```

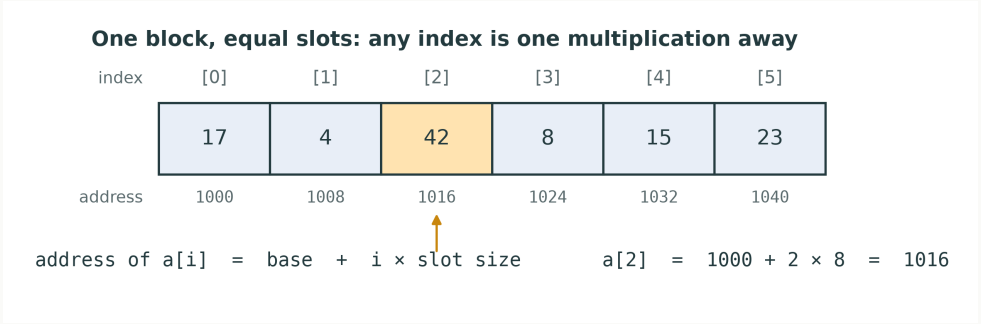
One line, **two hidden O(n) loops**: the slice **copies** up to n elements, and `in` **searches** them one by one. Inside an n-iteration loop: $O(n^2)$.

A call is only O(1) if you **know** it is O(1).

The Array

What an array is

A **fixed number** of **equal-sized slots**, stored **side by side** in memory.



$$\text{address of } a[i] = \text{base} + i \times \text{slot size}$$

One multiplication and one addition — **whatever i is, whatever n is.** That is why `a[i]` is $O(1)$.

In C, the array is the machine

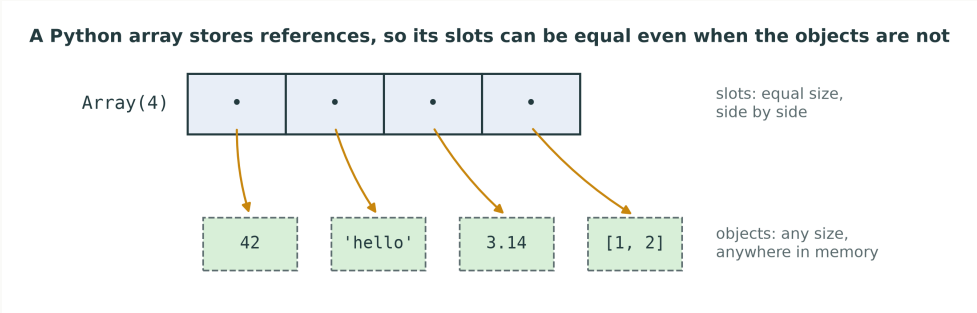
```
int a[6] = {17, 4, 42, 8, 15, 23};   // 6 × 4 bytes, reserved once
a[2] = 99;                           // base + 2 × 4: one instruction
a[6] = 0;                       // no check — writes over whatever is next
```

- The size is part of the type, fixed at compile time
- Every slot holds an `int`, so every slot is 4 bytes
- **No bounds check**: an out-of-range index is not an error, it is a bug

Arrays in Python: four candidates

	Grows?	Holds	Hides the cost?
`list`	yes — a dynamic array	references to any objects	**yes**: append, `insert`, `pop(0)`, `in`, slicing
`array.array`	yes	numbers only, one type, packed	yes: same methods as `list`
`numpy.ndarray`	no	numbers, one type, packed	yes: whole-array operations in C
`ctypes` array	**no**	C values, or references	**no** — index, assign, nothing else

Python stores references



A Python array slot does not hold 42; it holds **a reference to** the object 42. Slots are equal-sized — one reference each — even when the objects are not. That is how one array holds an int, a string and a list, and still indexes in O(1).

The Course Array

dsa/array.py — given to you

```
from dsa.array import Array

a = Array(5)                    # 5 slots, all None. Fixed for ever.
a[0] = 42                       # O(1)
a[0]                            # O(1) -> 42
len(a)                         # 5: the capacity, not "how many are used"
list(a)                         # iterate: [42, None, None, None, None]

Array.from_values([3, 1, 2])  # a full array, for tests and the REPL
Array(3, fill=0)               # every slot starts as 0
```

A real C array of references (`ctypes`), wrapped so that it **checks its bounds**.

What it will not do

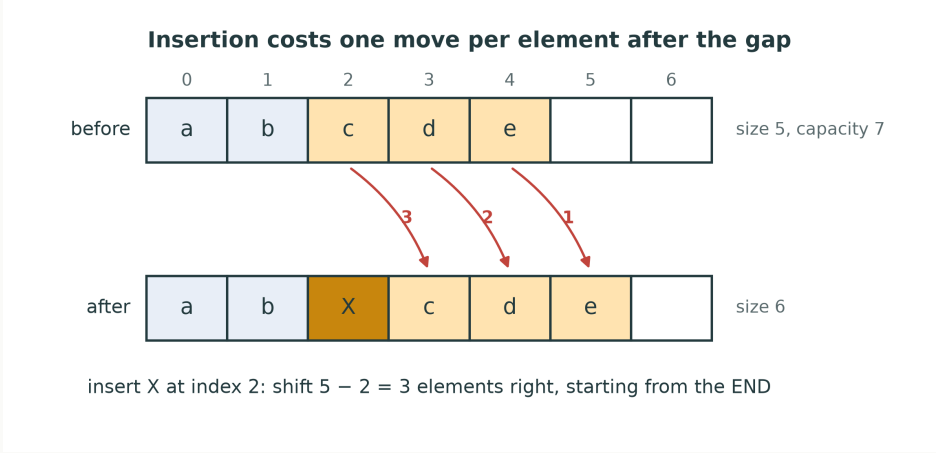
```
>>> a = Array(3)
>>> a[3]
IndexError: Array index 3 out of range for length 3 ...
>>> a[-1]
IndexError: ...                          # no negative indices, as in C
>>> a[0:2]
TypeError: Array does not support slicing; copy with a loop
>>> a.append(1)
AttributeError: 'Array' object has no attribute 'append'
```

No growing. No shrinking. No inserting. No slicing. No searching methods. **Every one of those is an algorithm — and you are going to write them.**

The operations, and what they cost

Operation	How	Cost
read / write slot `i`	address arithmetic	$O(1)$
find a value	check slots one by one	$O(n)$
insert at `i`	shift `i .. size-1` **right**, then write	$O(n)$
remove at `i`	shift `i+1 .. size-1` **left**	$O(n)$
insert / remove at the **end**	nothing to shift	$O(1)$
grow	new array, copy everything	$O(n)$

Insertion, slot by slot




```
def insert_at(arr, size, index, value):
    for i in range(size, index, -1):   # from the END, backwards
        arr[i] = arr[i - 1]
    arr[index] = value
```

Watch it run: the Array Lab

Array Lab — open it in any browser.

- The code, the variables and the **slots**, one step at a time
- Every **shift** animated; a counter of reads and writes
- `arr[i]`, `find`, `insert_at`, `remove_at`, `reverse_in_place`, `rotate_left`
- *What happens next?* — answer before you step
- Scenarios that **go wrong**: shifting from the front, a stale last slot, one step too far, `arr[-1]`

Measured, not asserted

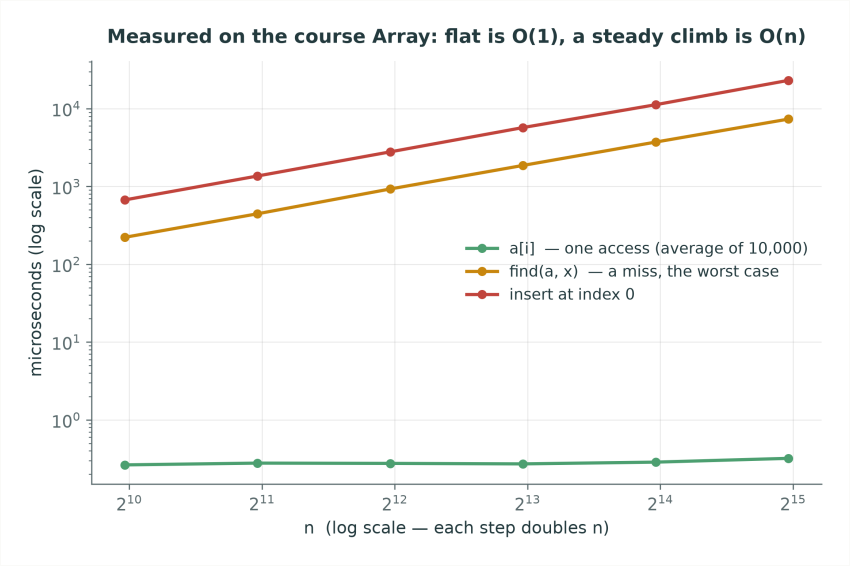
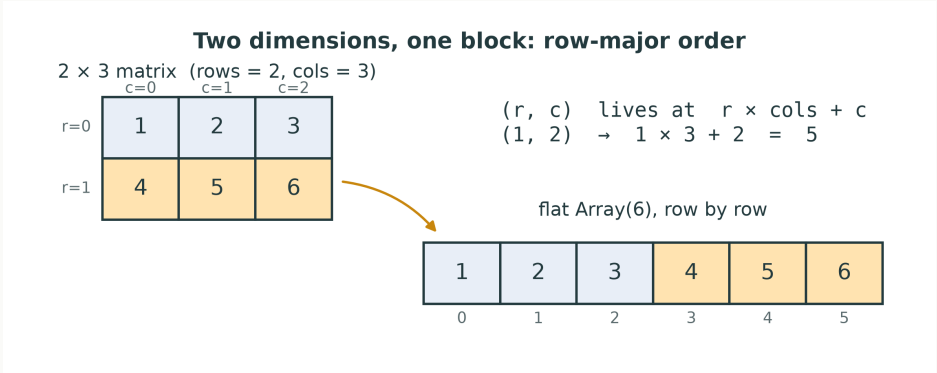


Figure: Real timings on the course `Array`. Each step right doubles n.

Two dimensions, one block



A matrix is stored as **one** array, row after row: element (r, c) lives at index $r \times \text{cols} + c$.

The Rule, From Today

No `list`, `dict` or `set` inside a structure

A data structure in `dsa/` keeps its data **only** in:

1. the course **`Array`** (`dsa/array.py`), or
2. **node objects** — `Node`, `TreeNode`, `Entry`, …, or
3. **another structure you have already built** in `dsa/`.

Never in a Python `list`, `dict`, `set` or `collections.deque`.

Lists remain the **interface**: functions may take a list and return a list, and the tests use them freely.

The same idea, all term

```
Array                              (given, O(1) index, fixed size)
 └── DynamicArray          (Week 4: grow by copying — amortised O(1) append)
       ├── Stack                   (Week 6)
       ├── SlowQueue               (Week 7 — pop(0) is O(n), measure it)
       └── MinHeap                 (Week 12)
             └── PriorityQueue
 ├── CircularQueue                 (Week 7 — O(1) both ends, on a ring)
 └── hash maps                     (Week 13)
       └── Graph                   (Week 14, with DynamicArray)
```

This Week

Exercises: `dsa/array_ops.py`

Function	Target
`find(arr, target, size=None)`	$O(n)$ worst, $O(1)$ best
`is_sorted(arr, size=None)`	$O(n)$, stop early
`insert_at(arr, size, index, value)`	$O(n)$ — shift right, from the end
`remove_at(arr, size, index)`	$O(n)$ — shift left
`reverse_in_place(arr)`	$O(n)$ time, $O(1)$ space
`rotate_left(arr, k)`	$O(n)$ time, $O(1)$ space — three reversals
`resized(arr, new_length)`	$O(n)$ — the dynamic array's one move
`merge_sorted(a, b)`	$O(n + m)$ — half of merge sort
`transpose_flat(arr, rows, cols)`	$O(rows \times cols)$

```
pytest tests/test_array.py -v          # the Array itself — passes already
pytest tests/test_array_ops.py -v        # your exercises
```

Homework 2 — before Lecture 03

1. **Implement** `dsa/array_ops.py` until `pytest tests/test_array_ops.py` passes.
2. **Measure** your `insert_at` at index 0 and at the end, for n = 1,000 … 32,000, with `viz.complexity.measure`, and plot both with `plot_growth(..., reference=["1", "n"])`. One paragraph: do the curves match the table?
3. **Prove** $5n^2 + 3n + 7 = O(n^2)$: give c and n_0 and show the inequality.
4. **Read off** the Big-O of the three snippets in the handout, and say which rule you used for each.
5. **Lab 02** exercises, if not finished in the lab.

Summary

Seven things to keep

1. Count **basic steps** as a function of n; keep the **shape**, drop the constants and the lower-order terms.
2. $f = O(g)$: **eventually** $f \leq c \cdot g$. Ω is $\geq$; Θ is both.
3. **Worst case** is the guarantee. Say which case you mean.
4. **Sequence adds, loops multiply, halving is log n** — and a call costs what the function costs.
5. An array is **contiguous, equal-sized slots**: $O(1)$ index, **fixed size**, $O(n)$ insert and search.
6. The course **`Array`**: index, assign, length. Nothing else.
7. From today: structures store data in **`Array`, nodes, or your own structures** — never `list`, `dict` or `set`.

Next

Week 3 — Recursion. Base case, recursive case, and the call stack — the first place where *how* you solve a problem decides its complexity.

Lab 03 this week ends with a bridge into it.